可持久化 DAG · 一份原文一张 DAG

节点 = 推导里的知识点；边 = 先懂什么才能懂什么；同一份原文的 DAG 建一次、写盘、之后读盘复用

例：一道简单的数学推导

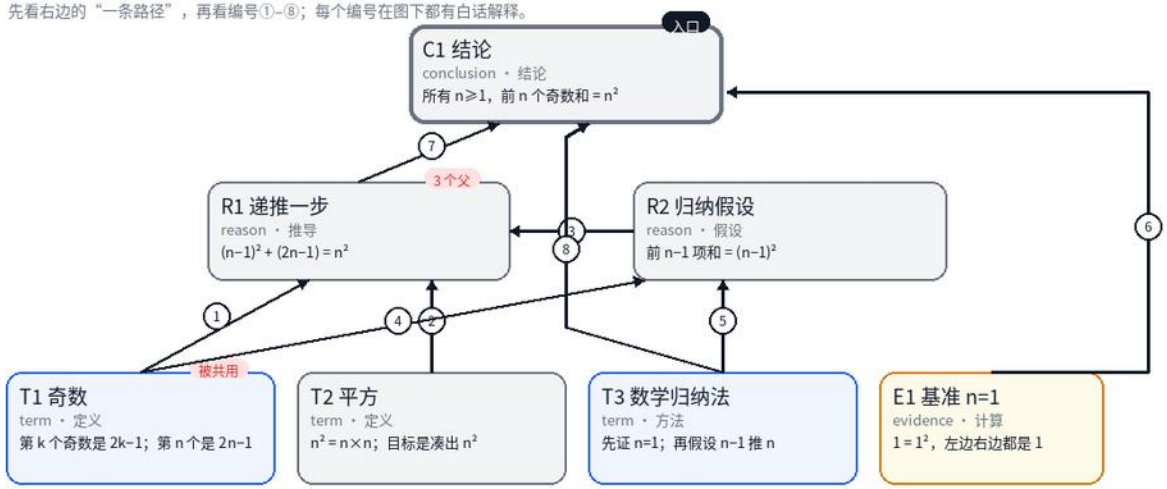
证明：$1+3+5+\cdots+(2n-1)=n^2$（$n\geq 1$）。基准 n=1：$1=1^2$；递推：若前 n−1 项和$=(n-1)^2$，加第 n 个奇数 $2n-1$ 后得 n^2。下面 7 个节点就是这两步拆出来的知识点。

关键一步：$(n-1)^2+(2n-1)=n^2-2n+1+2n-1=n^2$

下面 7 个节点就是“基准 + 递推”拆出来的知识点；每条边回答“为什么它在前”。

DAG 本体（节点里是推导的内容，不是代号）

先看右边的“一条路径”，再看编号①–⑧；每个编号在图下都有白话解释。



边上的编号，白话解释：

① T1 → R1	先把第 n 个奇数写成 2n−1	② T2 → R1	目标右边的 n^2 要会展开
③ R2 → R1	用归纳假设替换前 n−1 项	④ T1 → R2	前 n−1 项的最后一项是 2n−3
⑤ T3 → R2	归纳法允许“假设 n−1”	⑥ E1 → C1	基准情况 n=1 成立
⑦ R1 → C1	递推步成立	⑧ T3 → C1	两步合起来就是结论

怎么读：先走一条路径

1. T1 奇数 — 第 n 个奇数是 2n−1
2. R2 归纳假设 — 前 n−1 项和 = $(n-1)^2$
3. R1 递推一步 — 加第 n 个奇数，化简得 n^2
4. C1 结论 — 基准 + 递推，对所有 n≥1 成立

先定义 → 再假设 → 再递推 → 最后结论。

为什么不是树？

R1 的父是 T1（奇数）、T2（平方）、R2（归纳假设）；T1 又同时是 R1/R2 的前置。

树要求每个节点只有一个父；要表达共享前置，只能复制 T1。

复制会让同一个知识点出现两次，所以这里保留 DAG。

边数 > 节点数 − 1，也是“不是树”的直接标志。

节点颜色 = 节点类型

- 蓝 = term 定义
- 灰 = reason / conclusion
- 黄 = evidence 计算

持久化：不是每次重画，而是把这张图存下来

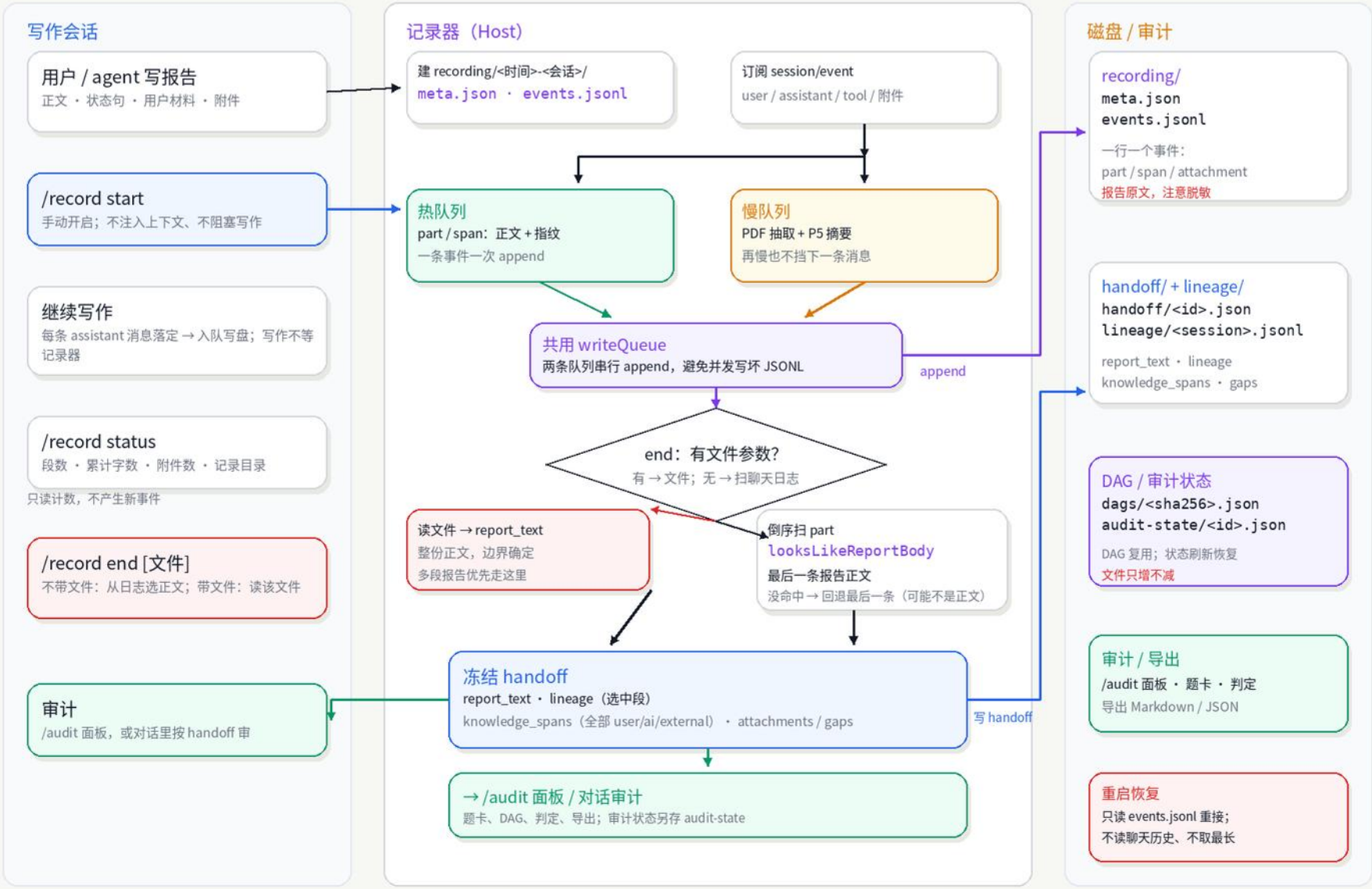
会话 A：选文件 → P1 建图 → 序列化写 dags/<hash>.json → 磁盘：graph / risk_points → 会话 B / 重启：同 hash 读盘 → /start 跳过 P1，只跑 P2

只存 graph / risk_points / 文件名 / 字数，不复制原文；同正文同 hash，原文改了就是新 DAG。

另存 audit-state/<audit_id>.json：队列、当前题、判定；刷新恢复，它不是 DAG 本身。

record 流程图 · 写作时落盘，结束时只读日志

手动开启 → 每条事件 append 到 events.jsonl → /record end 选正文 → handoff → 审计



边界：不注入上下文、不阻塞写作、不回看聊天历史；结束只读 events.jsonl。

正文选择是启发式：≥80 字、跳过短状态句、≥100 字；重要或多段报告用 /record end 文件.md。